

```

In [1]: import numpy as np
from scipy.optimize import minimize
import matplotlib.pyplot as plt

# =====
# 1. CORE PARAMETERS & BASE COSTS
# =====
batch_size = 50.0

names = ["Maize", "Soya Hipro", "Full Fat Soya", "Maize Gluten", "Rice ODGS", "Meat
base_costs = np.array([20.0, 60.0, 52.0, 36.0, 60.0, 40.0, 60.0, 100.0, 3.0, 45.0,

ME = np.array([3408, 3521, 2430, 2725, 3564, 2167, 2953, 8800, 0, 0, 0, 4000, 5000]
CP = np.array([8.18, 42.17, 51.78, 48.91, 62.77, 49.65, 57.39, 0, 0, 0, 0, 94.0, 58
Fat = np.array([4.18, 20.18, 1.73, 6.99, 6.88, 8.90, 99.13, 99.0, 0, 0, 0, 0, 0])
Ca = np.array([0.02, 0.15, 0.31, 0.31, 0.03, 12.50, 4.90, 0, 38.0, 22.0, 0, 0, 0])
P = np.array([0.05, 0.175, 0.24, 0.24, 0.07, 3.60, 0.84, 0, 0, 18.0, 0, 0, 0])
Lys = np.array([0.218, 2.107, 2.748, 1.161, 0.847, 1.415, 1.083, 0, 0, 0, 0, 78.0,
Met = np.array([0.158, 0.462, 0.606, 0.653, 1.359, 0.334, 0.592, 0, 0, 0, 0, 0, 99.
Na = np.array([0.02, 0.04, 0.02, 0.20, 0.01, 0.55, 0.45, 0, 0, 0, 39.3, 0, 0])
Cl = np.array([0.04, 0.03, 0.05, 0.45, 0.05, 0.69, 0.51, 0, 0, 0, 60.7, 0, 0])

nutrients = [CP, ME, Fat, Ca, P, Lys, Met, Na, Cl]
nutrient_names = ['Protein (%)', 'ME (kcal/kg)', 'Fat (%)', 'Calcium (%)', 'Phospho

# 3-Stage Nutritional Bounds [Pre-Starter, Starter, Finisher]
L_bounds = [
    np.array([23.0, 21.0, 20.0]), np.array([2950, 3000, 3050]), np.array([3.5, 3.5,
    np.array([0.9, 0.9, 0.92]), np.array([0.4, 0.45, 0.36]), np.array([1.2, 1.11, 1
    np.array([0.5, 0.6, 0.5]), np.array([0.2, 0.17, 0.17]), np.array([0.19, 0.17, 0
]
T_bounds = [
    np.array([24.0, 22.0, 21.0]), np.array([3000, 3050, 3150]), np.array([4.0, 4.0,
    np.array([7.0, 1.0, 1.0]), np.array([0.45, 0.46, 0.4]), np.array([1.3, 1.12, 1.
    np.array([0.6, 0.62, 0.59]), np.array([0.22, 0.22, 0.22]), np.array([0.24, 0.24
]

# Process Constraints & Ergonomics
steam_cost_kg, water_cost_kg, electricity_cost_kg = 40.0/1000, 2.9/1000, 138.0/1000
labor_cost_kg, packaging_cost_kg = 63.0/1000, 189.5/1000
worker_rwls = [17.02, 11.72, 6.56, 5.67, 10.35, 18.15]
worker_ids = ["PACKING 1", "PACKING 2", "LOADING 3", "LOADING 4", "LIFTING 5", "CUT

L_PDI, T_PDI_arr = np.array([85.0, 87.0, 86.0]), np.array([90.0, 92.0, 91.0])

# Bounds (Maize relaxed to 80%)
max_p = [0.80, 0.35, 0.30, 0.15, 0.15, 0.08, 0.10, 0.05, 0.03, 0.03, 0.01, 0.015, 0
bnds_single = [(0.001, batch_size * mp) for mp in max_p] + [(75, 85), (2, 4), (500,

# =====
# 2. MINITAB ADVANCED PDI ENGINE (16-TERM)
# =====
def calculate_pdi(T, M, PS, R):
    """Calculates PDI using the Minitab Box-Behnken Factorial Regression"""
    # Translation Layer: Convert physical limits to Minitab Coded (-1 to 1) Limits
    T_c = (T - 80.0) / 5.0
    M_c = (M - 3.0) / 1.0
    PS_c = (PS - 600.0) / 100.0
    R_c = (R - 9.0) / 3.0

    # 16-Term Thermodynamic Equation (Uncoded Coded-Hybrid)
    pdi = (

```

```

89.99
+ 1.837*T_c + 1.603*M_c - 0.2519*PS_c - 2.911*R_c
+ 1.798*T_c*M_c + 0.2431*T_c*PS_c - 1.048*T_c*R_c
- 0.2331*M_c*PS_c + 2.351*M_c*R_c - 1.957*PS_c*R_c
+ 2.342*T_c*M_c*PS_c - 1.542*T_c*M_c*R_c + 0.9356*T_c*PS_c*R_c
- 0.1181*M_c*PS_c*R_c + 0.7694*T_c*M_c*PS_c*R_c
)
return pdi

# =====
# 3. SOLVER ENGINE (LFGP)
# =====
def run_stage_solver(stage_idx, costs_array):
    def objective_single(vars):
        x = np.array(vars[0:13])
        T, M, PS, R, W = vars[13], vars[14], vars[15], vars[16], vars[17]

        # Priority 1: Nutrition
        nutr_penalty = 0
        nutr_sats = []
        for i in range(len(nutrients)):
            val = (x @ nutrients[i]) / batch_size
            L, U = L_bounds[i][stage_idx], T_bounds[i][stage_idx]
            if val < L: nutr_penalty += (L - val)**2 * 100000
            if val > U: nutr_penalty += (val - U)**2 * 100000
            sat = np.clip(1.0 - abs(val - ((L+U)/2)) / ((U-L)/2), 0, 1) if U > L else 1
            nutr_sats.append(sat)
        lam1 = np.mean(nutr_sats)

        # Priority 2: Pellet Quality (Calling the Minitab function)
        pdi_val = calculate_pdi(T, M, PS, R)
        lam2 = np.clip((pdi_val - L_PDI[stage_idx]) / (T_PDI_arr[stage_idx] - L_PDI[stage_idx]), 0, 1)

        # Priority 3: Cost
        c_tot = np.sum(x * costs_array) + (steam_cost_kg*T*batch_size + water_cost_kg*W*batch_size)
        lam3 = np.clip((2400.0 - c_tot) / (2400.0 - 2100.0), 0, 1)

        # Priority 4: Ergonomics
        lam4 = np.clip((1.8 - np.mean([W / rw1 for rw1 in worker_rwls])) / (1.8 - 0.5), 0, 1)

        return -(10000*lam1 + 1000*lam2 + 100*lam3 + 10*lam4) + nutr_penalty

    cons = [{'type': 'eq', 'fun': lambda v: np.sum(v[0:13]) - batch_size}]
    guess = [30.0, 5.0, 5.0, 2.0, 2.0, 2.0, 2.0, 0.5, 0.5, 0.5, 0.1, 0.2, 0.2]
    guess = [g * (batch_size / sum(guess)) for g in guess] + [80.0, 3.0, 600.0, 8.0]

    return minimize(objective_single, guess, method='SLSQP', bounds=bnds_single, constraints=cons)

# =====
# 4. RUN BASELINE OPTIMIZATION
# =====
x_opt = np.zeros((3, 13))
T_opt, M_opt, PS_opt, R_opt, W_opt, a_PDI = np.zeros(3), np.zeros(3), np.zeros(3), np.zeros(3), np.zeros(3)
total_cost_all = 0
stages_names = ['Pre-Starter', 'Starter', 'Finisher']

for s in range(3):
    res = run_stage_solver(s, base_costs)
    x_opt[s] = res.x[0:13]
    T_opt[s], M_opt[s], PS_opt[s], R_opt[s], W_opt[s] = res.x[13:18]
    a_PDI[s] = calculate_pdi(T_opt[s], M_opt[s], PS_opt[s], R_opt[s])

    c_tot = np.sum(x_opt[s] * base_costs) + (steam_cost_kg*T_opt[s]*batch_size + water_cost_kg*W_opt[s]*batch_size)
    total_cost_all += c_tot

```

```

# =====
# 5. EXPANDED LIFECYCLE SENSITIVITY
# =====
print("\n" + "#" * 80)
print(" LIFECYCLE SHADOW PRICING & COMMODITY RISK ANALYSIS")
print("#" * 80)

commodity_shocks = [
    (0, "Maize", [15.0, 20.0, 25.0, 30.0, 35.0]),
    (1, "Soya Hipro", [50.0, 60.0, 70.0, 80.0, 90.0]),
    (2, "Full Fat Soya", [40.0, 52.0, 64.0, 76.0, 88.0]),
    (7, "Soya Oil", [80.0, 100.0, 120.0, 140.0, 160.0])
]

sensitivity_results = {s: {c[1]: {'prices': c[2], 'costs': [], 'inclusion': []}} for

for s_idx, stage in enumerate(stages_names):
    print(f"\n" + "=" * 60)
    print(f" EVALUATING STAGE: {stage.upper()}")
    print("=" * 60)

    for (ing_idx, ing_name, test_prices) in commodity_shocks:
        print(f"\n--- Shocking {ing_name} Market Price ---")

        for price in test_prices:
            test_costs = base_costs.copy()
            test_costs[ing_idx] = price

            res_sens = run_stage_solver(s_idx, test_costs)
            ing_used = res_sens.x[ing_idx]
            T, M, PS, R, W = res_sens.x[13:18]

            c_tot = np.sum(res_sens.x[0:13] * test_costs) + (steam_cost_kg * T * batch_

            sensitivity_results[stage][ing_name]['costs'].append(c_tot)
            sensitivity_results[stage][ing_name]['inclusion'].append(ing_used)

            print(f"Rs. {price:<6.2f} | Inc: {ing_used:>5.2f}kg | Stage Cost: Rs. {

# =====
# 6. MASTER DASHBOARD PLOTTING
# =====
# Part A: Lifecycle Risk Plot
fig1, axs = plt.subplots(1, 3, figsize=(20, 6), sharey=False)
fig1.suptitle('Lifecycle Commodity Risk Exposure: Total Cost Impact per Stage', for
colors_plot = ['#f1c40f', '#e74c3c', '#e67e22', '#3498db']
markers = ['o', 's', '^', 'D']

for s_idx, stage in enumerate(stages_names):
    ax = axs[s_idx]
    for c_idx, (ing_idx, ing_name, _) in enumerate(commodity_shocks):
        base_price = base_costs[ing_idx]
        prices = np.array(sensitivity_results[stage][ing_name]['prices'])
        price_shock_pct = ((prices - base_price) / base_price) * 100
        costs = sensitivity_results[stage][ing_name]['costs']
        ax.plot(price_shock_pct, costs, marker=markers[c_idx], color=colors_plot[c_

    ax.set_title(f'{stage} Stage', fontweight='bold', fontsize=14)
    ax.set_xlabel('Commodity Price Shock (%)', fontweight='bold')
    ax.set_ylabel('Optimized Total Stage Cost (Rs)', fontweight='bold')
    ax.grid(True, linestyle='--', alpha=0.6)
    ax.axvline(x=0, color='black', linestyle='-', alpha=0.3)

```

```

axs[2].legend(title='Commodity Shocked', loc='center left', bbox_to_anchor=(1, 0.5))
plt.tight_layout(rect=[0, 0, 0.92, 1])

# Part B: Base Optimizations
fig2, axs2 = plt.subplots(2, 3, figsize=(20, 12))
fig2.suptitle('Broiler Feed Formulation Dashboard (Baseline Results)', fontsize=18,
pie_colors = plt.cm.tab20(np.linspace(0, 1, 13))

for s in range(3):
    labels = [names[i] for i in range(13) if x_opt[s, i] > 0.1]
    sizes = [x_opt[s, i] for i in range(13) if x_opt[s, i] > 0.1]
    cols = [pie_colors[i] for i in range(13) if x_opt[s, i] > 0.1]
    axs2[0, s].pie(sizes, labels=labels, colors=cols, autopct='%1.1f%%', startangle=
    axs2[0, s].set_title(f'{stages_names[s]}\nRecipe Formulation', fontweight='bold')

c_raw_all = np.sum(x_opt @ base_costs)
c_proc_all = np.sum(steam_cost_kg*T_opt*batch_size + water_cost_kg*M_opt*batch_size)
c_lab_all = np.sum(labor_cost_kg * batch_size * (10 / np.maximum(1, R_opt)))
c_pack_all = np.sum(packaging_cost_kg * W_opt)
cost_vals = [c_raw_all, c_proc_all, c_lab_all, c_pack_all]
axs2[1, 0].pie(cost_vals, labels=['Raw Material', 'Energy', 'Labor', 'Packaging'],
axs2[1, 0].set_title('Global Cost Breakdown', fontweight='bold')

axs2[1, 1].bar(['Temp (°C)', 'Moist (%)', 'Size (µm/10)', 'Rate (t/h)'], [np.mean(T
axs2[1, 1].set_title('Average Machine Settings', fontweight='bold')

li_scores = [np.mean([W_opt[s] / worker_rwls[i] for s in range(3)]) for i in range(
bar_colors = ['#2ecc71' if li<=0.8 else '#f1c40f' if li<=1.0 else '#e67e22' if li<=
bars_li = axs2[1, 2].bar(worker_ids, li_scores, color=bar_colors, edgecolor='black')
axs2[1, 2].axhline(0.8, color='green', linestyle='--', label='Ideal (0.8)')
axs2[1, 2].axhline(1.2, color='red', linestyle='--', label='Max (1.2)')
axs2[1, 2].set_ylabel('Lifting Index (LI)')
axs2[1, 2].set_title(f'Worker Lifting Risk (Avg Bag = {np.mean(W_opt):.1f}kg)', for
axs2[1, 2].legend()
for bar in bars_li:
    axs2[1, 2].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.05, f'{ba

fig2.tight_layout()
plt.show()

```


LIFECYCLE SHADOW PRICING & COMMODITY RISK ANALYSIS
#####

=====

EVALUATING STAGE: PRE-STARTER

=====

--- Shocking Maize Market Price ---

Rs. 15.00		Inc: 26.49kg		Stage Cost: Rs. 1770.96
Rs. 20.00		Inc: 25.43kg		Stage Cost: Rs. 1895.10
Rs. 25.00		Inc: 25.30kg		Stage Cost: Rs. 2022.46
Rs. 30.00		Inc: 30.00kg		Stage Cost: Rs. 2185.35
Rs. 35.00		Inc: 30.00kg		Stage Cost: Rs. 2335.35

--- Shocking Soya Hipro Market Price ---

Rs. 50.00		Inc: 4.25kg		Stage Cost: Rs. 1866.18
Rs. 60.00		Inc: 4.24kg		Stage Cost: Rs. 1895.10
Rs. 70.00		Inc: 4.18kg		Stage Cost: Rs. 1948.48
Rs. 80.00		Inc: 4.14kg		Stage Cost: Rs. 2000.24
Rs. 90.00		Inc: 4.17kg		Stage Cost: Rs. 2033.63

--- Shocking Full Fat Soya Market Price ---

Rs. 40.00		Inc: 6.55kg		Stage Cost: Rs. 1842.38
Rs. 52.00		Inc: 6.52kg		Stage Cost: Rs. 1895.10
Rs. 64.00		Inc: 6.51kg		Stage Cost: Rs. 1986.66
Rs. 76.00		Inc: 6.19kg		Stage Cost: Rs. 2029.58
Rs. 88.00		Inc: 6.54kg		Stage Cost: Rs. 2142.07

--- Shocking Soya Oil Market Price ---

Rs. 80.00		Inc: 0.65kg		Stage Cost: Rs. 1902.66
Rs. 100.00		Inc: 0.58kg		Stage Cost: Rs. 1895.10
Rs. 120.00		Inc: 0.58kg		Stage Cost: Rs. 1907.05
Rs. 140.00		Inc: 0.61kg		Stage Cost: Rs. 1933.17
Rs. 160.00		Inc: 0.61kg		Stage Cost: Rs. 1945.46

=====

EVALUATING STAGE: STARTER

=====

--- Shocking Maize Market Price ---

Rs. 15.00		Inc: 30.00kg		Stage Cost: Rs. 1735.35
Rs. 20.00		Inc: 30.00kg		Stage Cost: Rs. 1885.35
Rs. 25.00		Inc: 30.00kg		Stage Cost: Rs. 2035.35
Rs. 30.00		Inc: 26.50kg		Stage Cost: Rs. 2154.95
Rs. 35.00		Inc: 26.79kg		Stage Cost: Rs. 2281.83

--- Shocking Soya Hipro Market Price ---

Rs. 50.00		Inc: 5.00kg		Stage Cost: Rs. 1835.35
Rs. 60.00		Inc: 5.00kg		Stage Cost: Rs. 1885.35
Rs. 70.00		Inc: 5.00kg		Stage Cost: Rs. 1935.35
Rs. 80.00		Inc: 5.00kg		Stage Cost: Rs. 1985.35
Rs. 90.00		Inc: 5.00kg		Stage Cost: Rs. 2035.35

--- Shocking Full Fat Soya Market Price ---

Rs. 40.00		Inc: 5.00kg		Stage Cost: Rs. 1825.35
Rs. 52.00		Inc: 5.00kg		Stage Cost: Rs. 1885.35
Rs. 64.00		Inc: 5.00kg		Stage Cost: Rs. 1945.35
Rs. 76.00		Inc: 5.00kg		Stage Cost: Rs. 2005.35
Rs. 88.00		Inc: 5.00kg		Stage Cost: Rs. 2065.35

--- Shocking Soya Oil Market Price ---

Rs. 80.00		Inc: 0.50kg		Stage Cost: Rs. 1875.35
Rs. 100.00		Inc: 0.50kg		Stage Cost: Rs. 1885.35

Rs. 120.00	Inc: 0.50kg	Stage Cost: Rs. 1895.35
Rs. 140.00	Inc: 0.50kg	Stage Cost: Rs. 1905.35
Rs. 160.00	Inc: 0.50kg	Stage Cost: Rs. 1915.35

=====

EVALUATING STAGE: FINISHER

=====

--- Shocking Maize Market Price ---

Rs. 15.00	Inc: 29.43kg	Stage Cost: Rs. 1736.11
Rs. 20.00	Inc: 29.43kg	Stage Cost: Rs. 1883.25
Rs. 25.00	Inc: 29.43kg	Stage Cost: Rs. 2030.39
Rs. 30.00	Inc: 28.34kg	Stage Cost: Rs. 2181.28
Rs. 35.00	Inc: 28.92kg	Stage Cost: Rs. 2324.66

--- Shocking Soya Hipro Market Price ---

Rs. 50.00	Inc: 4.90kg	Stage Cost: Rs. 1834.20
Rs. 60.00	Inc: 4.90kg	Stage Cost: Rs. 1883.25
Rs. 70.00	Inc: 4.90kg	Stage Cost: Rs. 1932.30
Rs. 80.00	Inc: 4.90kg	Stage Cost: Rs. 1981.35
Rs. 90.00	Inc: 4.90kg	Stage Cost: Rs. 2030.39

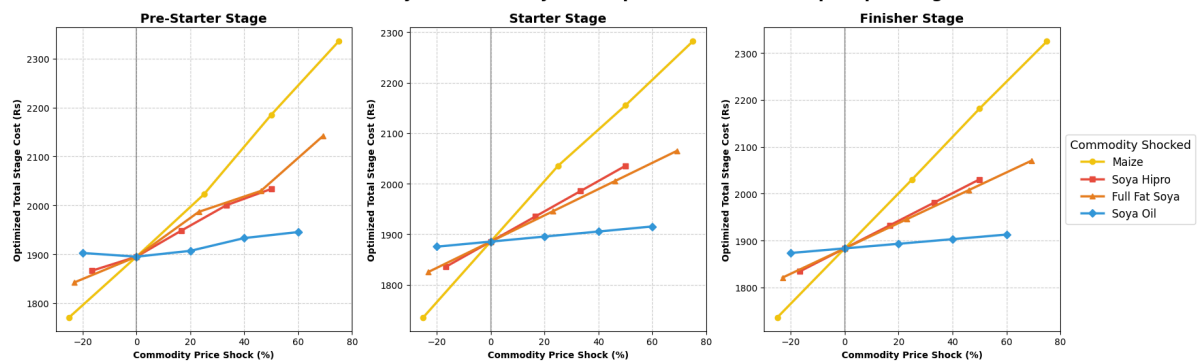
--- Shocking Full Fat Soya Market Price ---

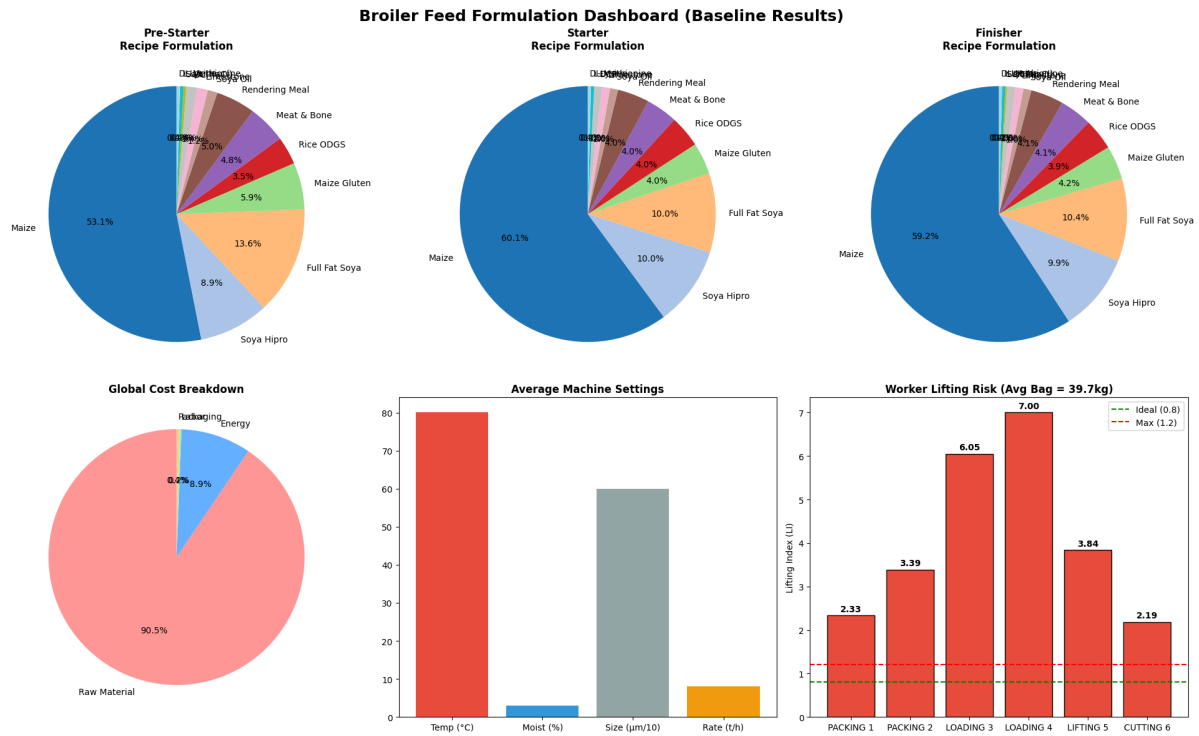
Rs. 40.00	Inc: 5.19kg	Stage Cost: Rs. 1820.96
Rs. 52.00	Inc: 5.19kg	Stage Cost: Rs. 1883.25
Rs. 64.00	Inc: 5.19kg	Stage Cost: Rs. 1945.54
Rs. 76.00	Inc: 5.19kg	Stage Cost: Rs. 2007.83
Rs. 88.00	Inc: 5.19kg	Stage Cost: Rs. 2070.11

--- Shocking Soya Oil Market Price ---

Rs. 80.00	Inc: 0.49kg	Stage Cost: Rs. 1873.44
Rs. 100.00	Inc: 0.49kg	Stage Cost: Rs. 1883.25
Rs. 120.00	Inc: 0.49kg	Stage Cost: Rs. 1893.06
Rs. 140.00	Inc: 0.49kg	Stage Cost: Rs. 1902.87
Rs. 160.00	Inc: 0.49kg	Stage Cost: Rs. 1912.68

Lifecycle Commodity Risk Exposure: Total Cost Impact per Stage





```
In [ ]:
```